seatlock: Java/Spring Boot vs Rust

Same API, same PostgreSQL, same k6 load, same container limits (2 CPU, 1 GiB). Memory is the resident set (VmRSS) of the service process, sampled every second. Medians across valid runs.

Memory at idle
4.4 MiB
vs 276 MiB for Java (tuned)
· 98% lower

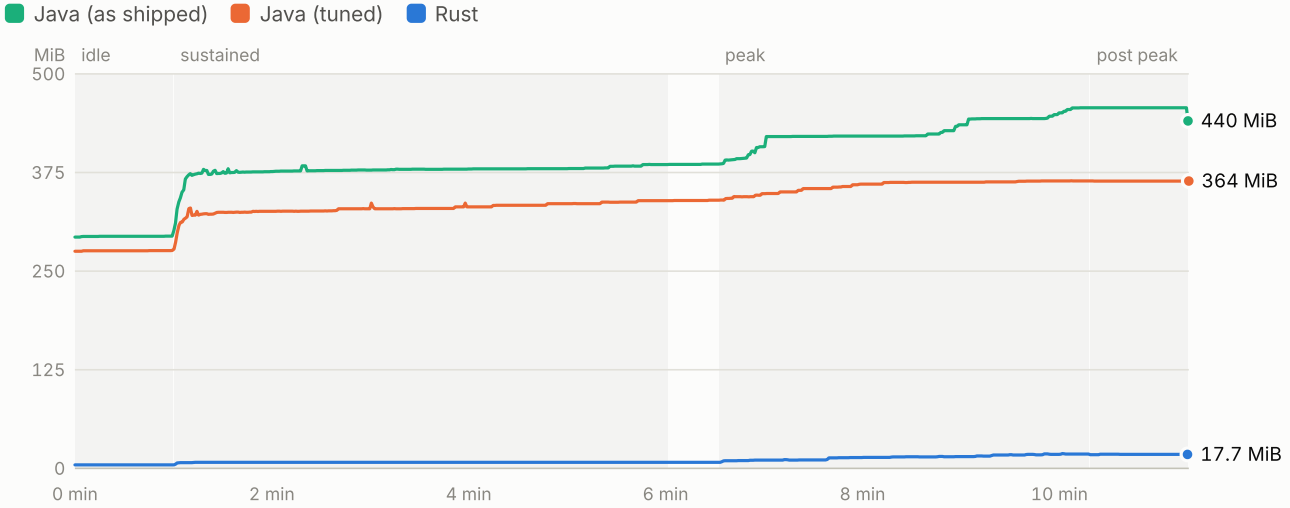
Memory under load
7.8 MiB
vs 333 MiB for Java (tuned)
· 98% lower

Memory at peak
18.9 MiB
vs 367 MiB for Java (tuned)
· 95% lower

p99 latency under load
5.6 ms
vs 44.3 ms for Java (tuned)
· 87% lower

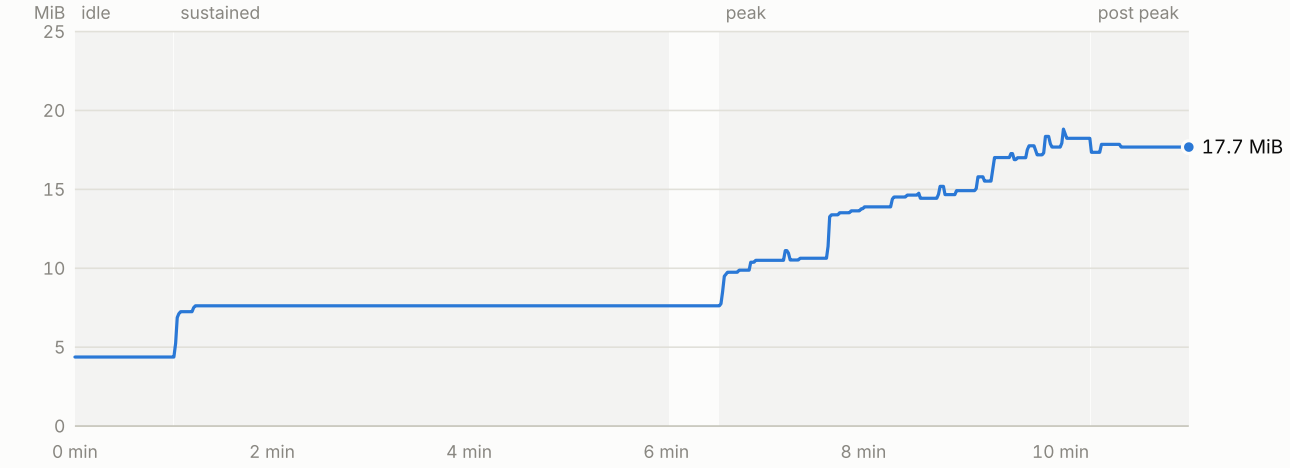
Memory over a full benchmark run

Idle, then 5 minutes at ~415 req/s, a cooldown, a flash-sale burst and a ramp to saturation, then idle again. Java keeps its peak memory afterwards.



Rust alone, on its own scale

At the scale above, Rust is a flat line near zero. Here is the same run zoomed in.



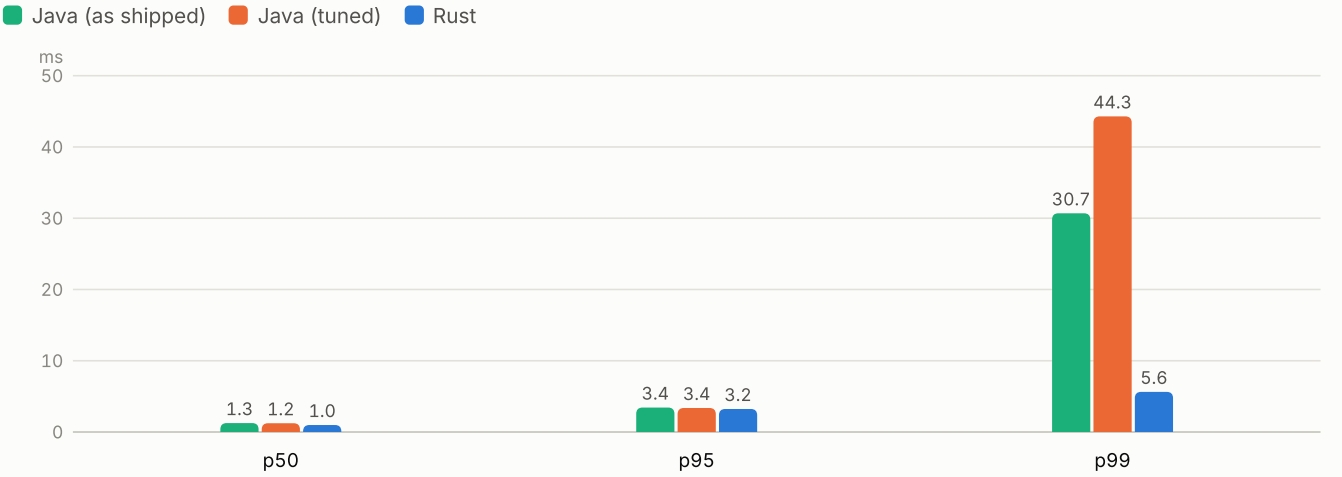
Memory by phase





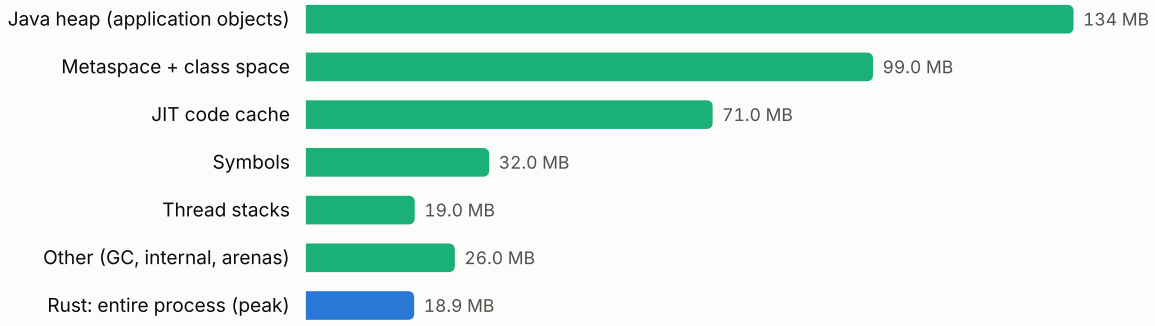
Latency at the same offered load (sustained phase)

Garbage-collection pauses show up in Java's p99; Rust has no collector.



Where the JVM's memory goes

Java as shipped, after 2 minutes of load (Native Memory Tracking, committed MB). Only the heap holds the service's data; the rest is runtime the JVM pays regardless. Rust's whole process is shown for scale.



All numbers

Metric	Java (as shipped)	Java (tuned)	Rust
Runs	3	3	3
RSS idle (MiB)	294	276	4.4
RSS sustained p50 (MiB)	383	333	7.8
RSS peak (MiB)	457	367	18.9
RSS after peak (MiB)	457	365	16.7

Threads at peak	222	72	3
Startup to healthy (s)	4	5	0
Sustained p50 (ms)	1.25	1.23	0.98
Sustained p95 (ms)	3.43	3.37	3.23
Sustained p99 (ms)	30.69	44.29	5.63
Sustained throughput (req/s)	413	413	415
Peak throughput (req/s)	1,887	1,894	1,902

Timeline charts plot each target's representative run (median peak): Java (as shipped) run1, Java (tuned) run1, Rust run1.